Midterm: #2 - Penetration Testing

ENPM634-0101,CY01: Penetration Testing

Nelay Sharma, Sheldon Douglas, Aron Sipos

Mr. Michael Wittner

Date: 4/28/2026

Directory ID: nsharm21, sdoug13, asipos1

University ID: #122295414 #117865974, #117258134

Word Count: 1925

Table of Contents

Stage	Name	Flag
#1	SQL Injection Bypass	ENPM634{sql_injection_login_bypass}
#2	Server-Side File Upload Execution	ENPM634{file_upload_server_side_exec}
#3	Insecure Direct Object Reference	ENPM634{idor_private_question_access}
#4	JWT Token Forgery	ENPM634{weak_jwt_secret_admin_forgery}

[This page is Intentionally left blank]

Flag #1 - Challenge Summary

Name: SQL Injection Bypass
Category: Web
Severity: High
Description: The DevFlow login form is vulnerable to SQL injection via username input.

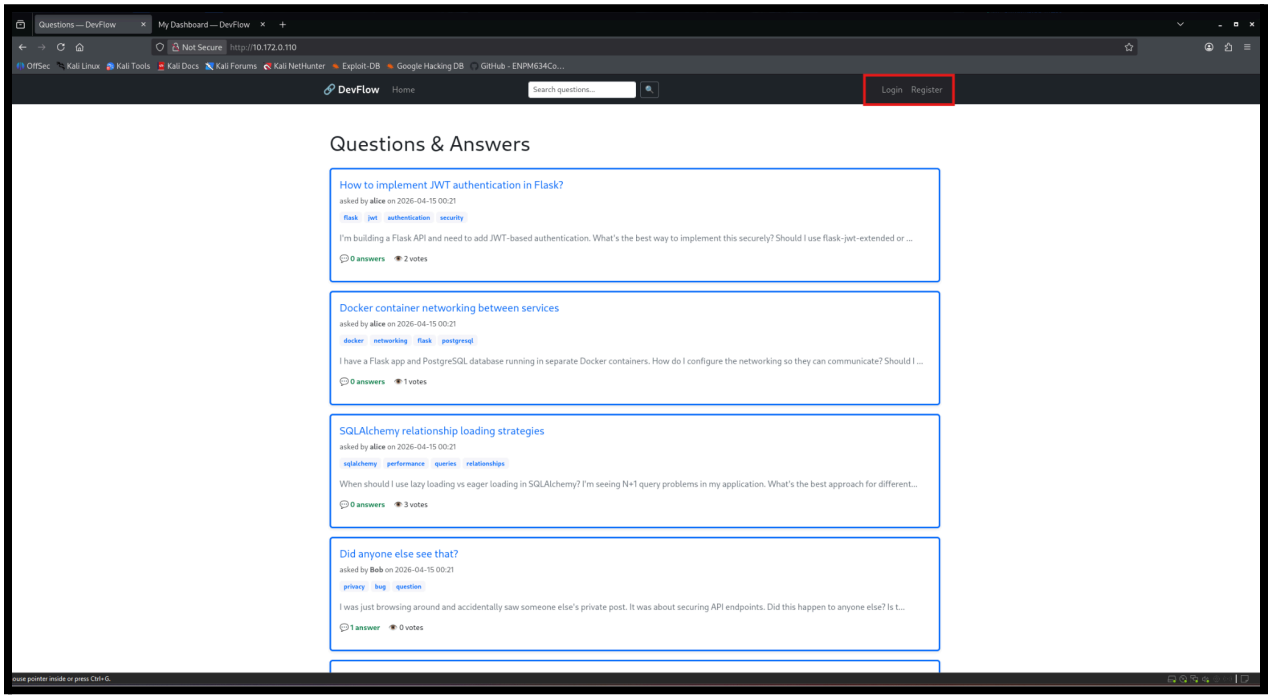
Steps to Reproduce

(1) After spinning up the VM environment on the `playground.ragnarsecurity` site, we then in our VM environment entered `10.172.0.110` into our Kali Firefox and was presented with a DevFlow playground environment.

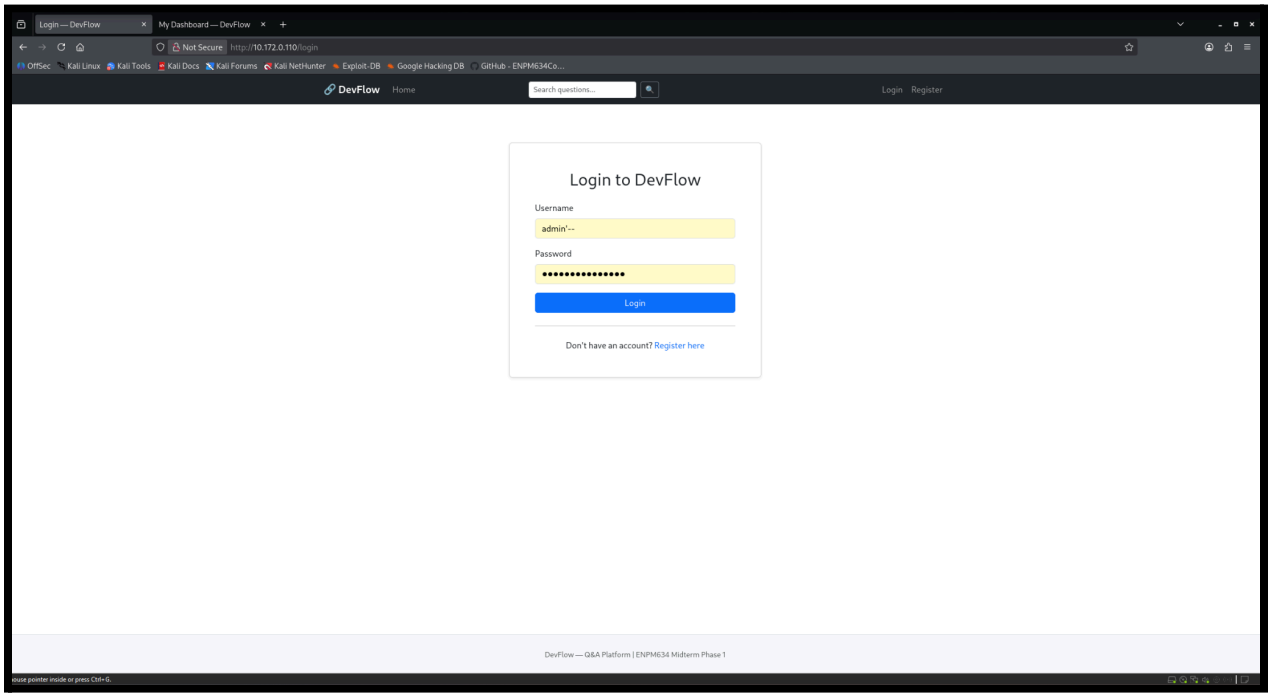
(2) Using the login page at `http://10.172.0.110/login`, we entered `admin'--` into the Username field and entered obfuscated text for Password field to see if there were any input validation detectors for mimicking a real password to bypass any potential client-side length or pattern filters

(3) Click Login Button; Observing the application grants an authenticated session as 'Bob' and displays the success banner with the proper flag: `ENPM634{sql_injection_login_bypass}`.

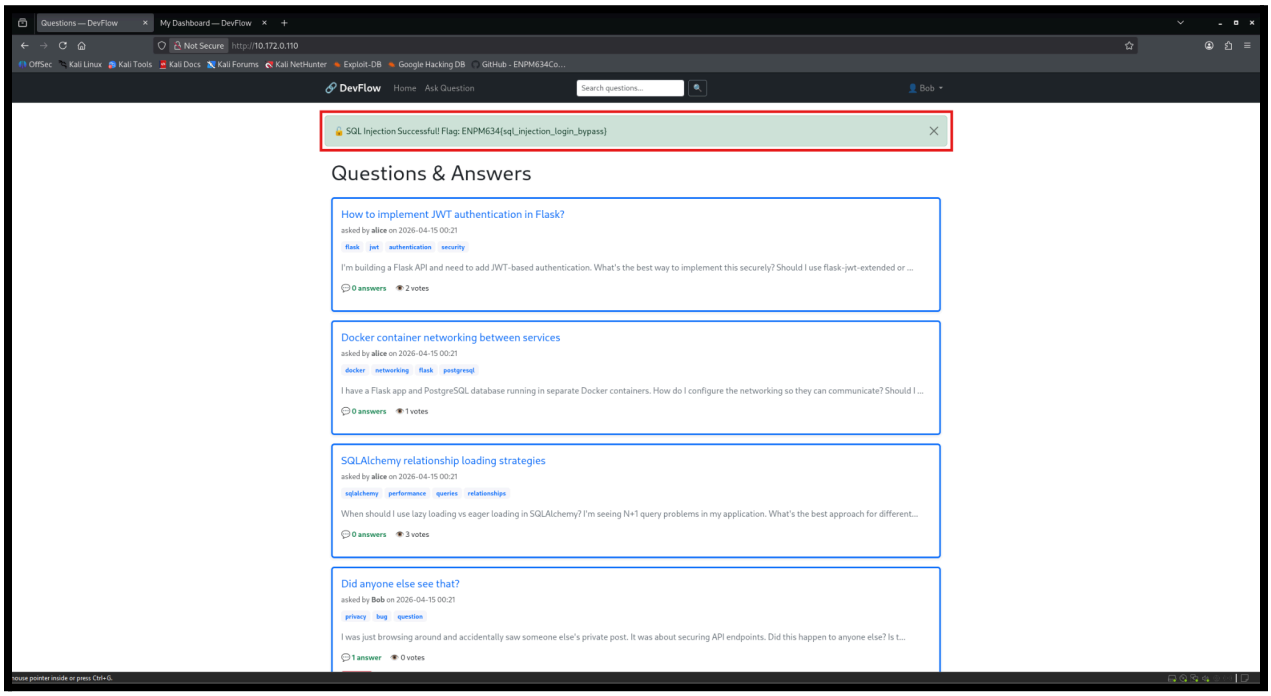
Screenshot #1: Unauthenticated home page view



Screenshot #2: Login form with SQL Injection payload



Screenshot #1: Successful admin bypass confirmed



Impact

An attacker can bypass the login and gain access to any account, including administrative accounts without knowing the password. In this case, the exploit resulted in a fully authenticated session as ‘Bob’, exposing privileged functionality, user data, and any content restricted to authenticated session users. On a real system, this could lead to full account takeover and unauthorized access to sensitive user data.

Remediation

The cause is a direct string interpolation of user input into SQL queries (i.e., pasting user input directly into SQL). The fix for DevFlow is to replace the query construction with parameterized queries (i.e., also called prepared statements) to ensure user input is always treated as data and never as executable SQL. In laymen terms, use placeholders, not string glue.

Tools: Kali Linux, Firefox Browser, SQL

SQL Injection Code

```
admin'--
DbeygTSKQNtyftnXsKYBFyxPFnMdVKwhAFtzGy
```

Flag #2 - Challenge Summary

Name: Server-Side File Upload Execution (Remote Code Execution via Python Upload)
Category: Web
Severity: Critical
Description: The DevFlow feature allows users to attach and execute files server-side.

Steps to Reproduce

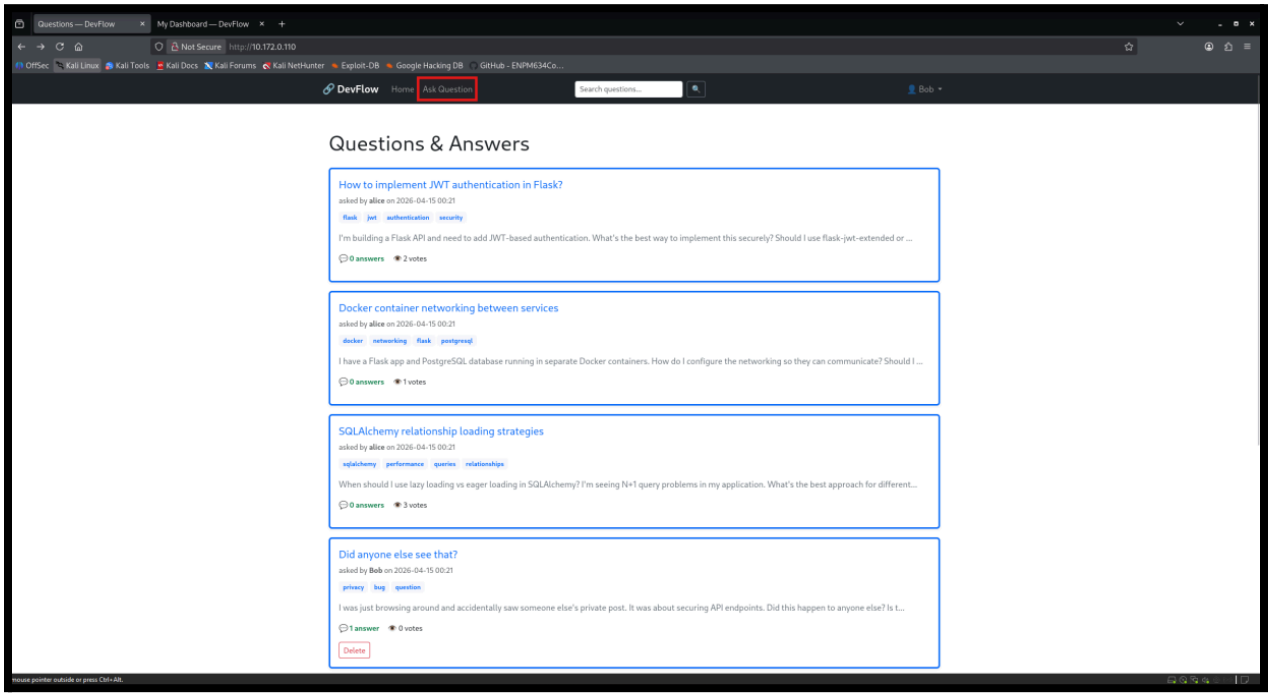
(1) Directly leaving off after we gained access from 'Bob' in Flag #1, we navigated to the `Ask a Question` page via the navbar.

(2) We crafted a Python script (i.e., '`:P.py`') containing `os.environ` enumeration to probe whether uploaded files were executed server-side.

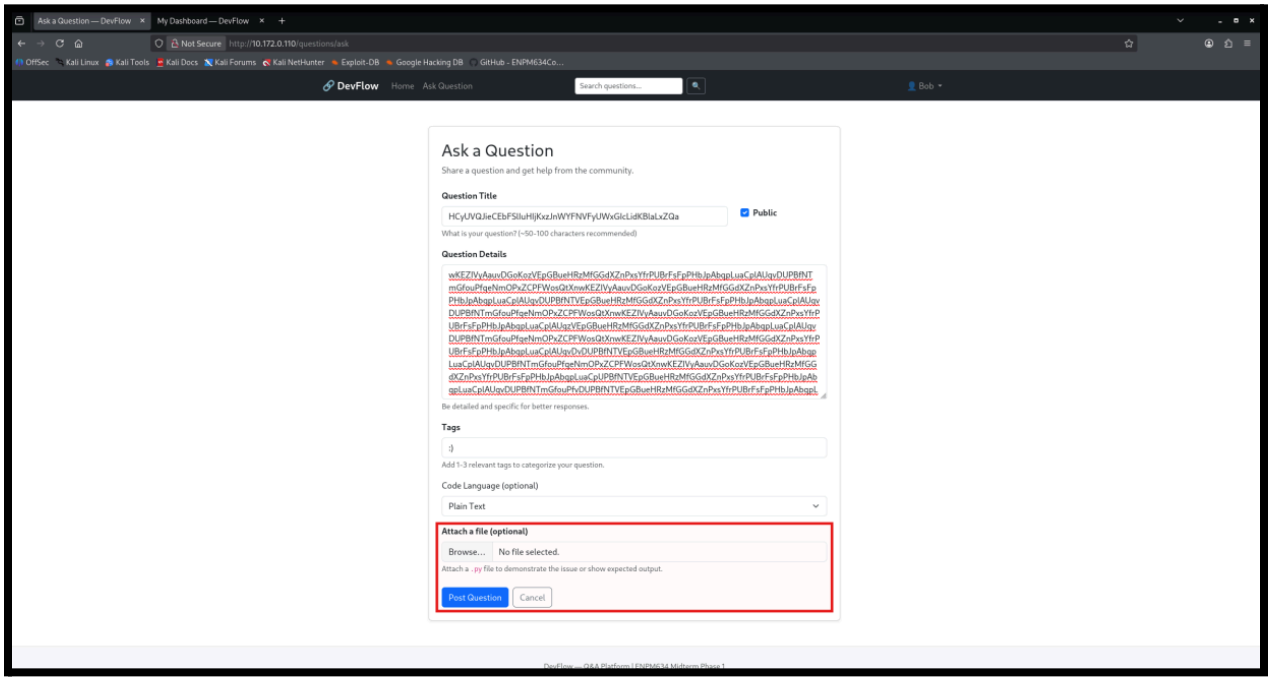
(3) We filled in randomized string values for the Question Title, Question Details, and Tags fields, then attached '`:P.py`' using the `Browse file` upload field, the length and content was done to see if there were any input validation detectors.

(4) We clicked Post Question and observed the success banner; where we observed the flag: `ENPM634{file_upload_server_side_exec}`

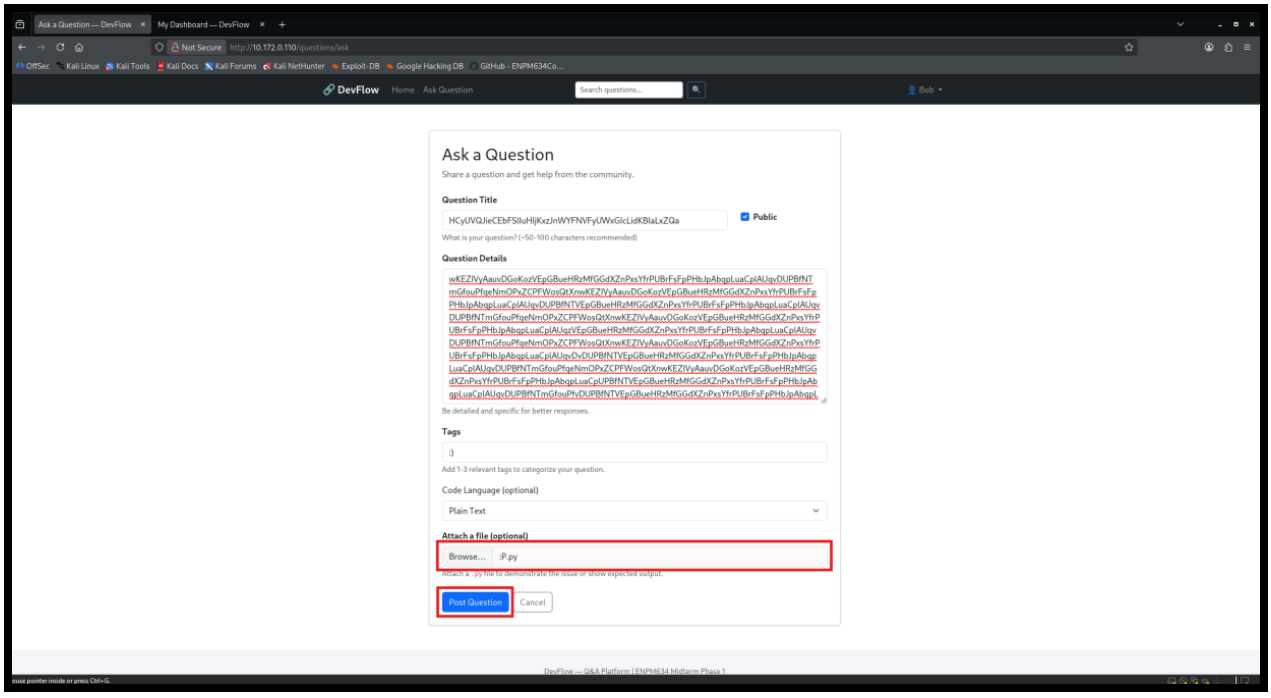
Screenshot #1: Authenticated home page, Ask Question



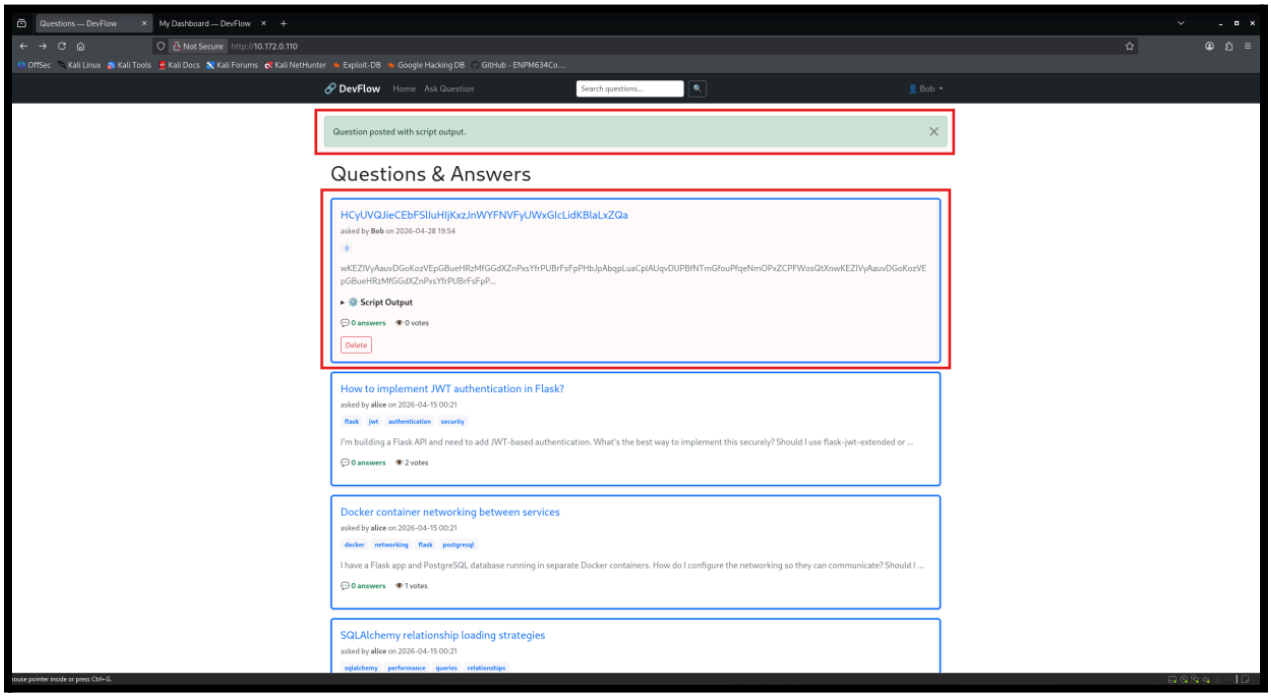
Screenshot #2: Question form with payload attached



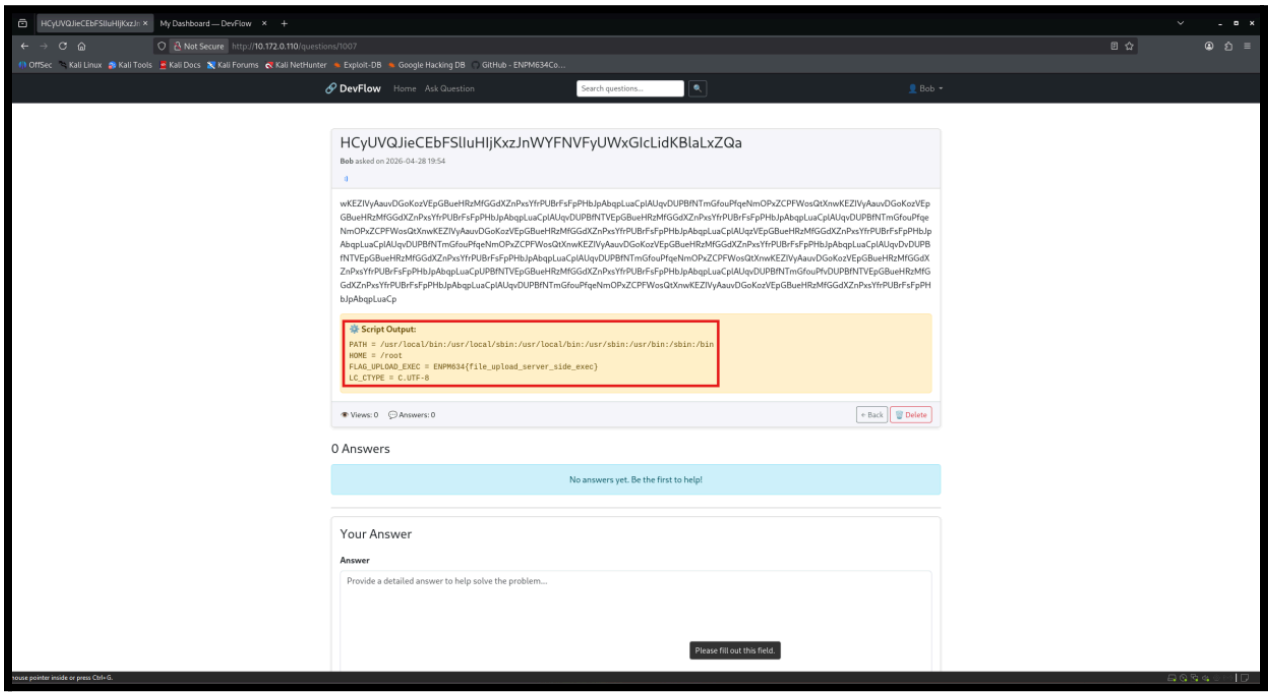
Screenshot #3: File upload field showing :P.py



Screenshot #4: Success banner, question posted



Screenshot #5: Script output with flag exposed



Impact

An attacker can execute arbitrary Python code on the server under the application's runtime context, thereby gaining full control of the server environment . In this case, full server environment variables were dumped, exposing a sensitive flag and system paths. On a real system this could lead to credential theft, lateral movement, full server compromise, or data exfiltration.

Remediation

The application should never execute uploaded files. File uploads should be restricted to static content types (i.e., images, PDFs, plain text files, CSV spreadsheets) with strict MIME type and extension validation to reject executable or scripted file types. Uploaded files must be stored outside the web root with no execution permissions. In addition, any uploaded files should be renamed server-side using randomly generated (i.e., UUID-based names like `a3f92c1d-4e78-11ef.pdf` or `7b21fc09-9a3b-42dd.png`) to prevent path-based execution tricks.

Tools: Kali Linux, Firefox Browser, Python

':P.py' Source Code

```
import os

for key, val in os.environ.items():
    print(f"{key} = {val}")
```

Flag #3 - Challenge Summary

Name: Insecure Direct Object Reference (IDOR) on Private Questions
Category: Web
Severity: High
Description: The DevFlow exposes information via predictable sequential integer IDs in URL.

Steps to Reproduce

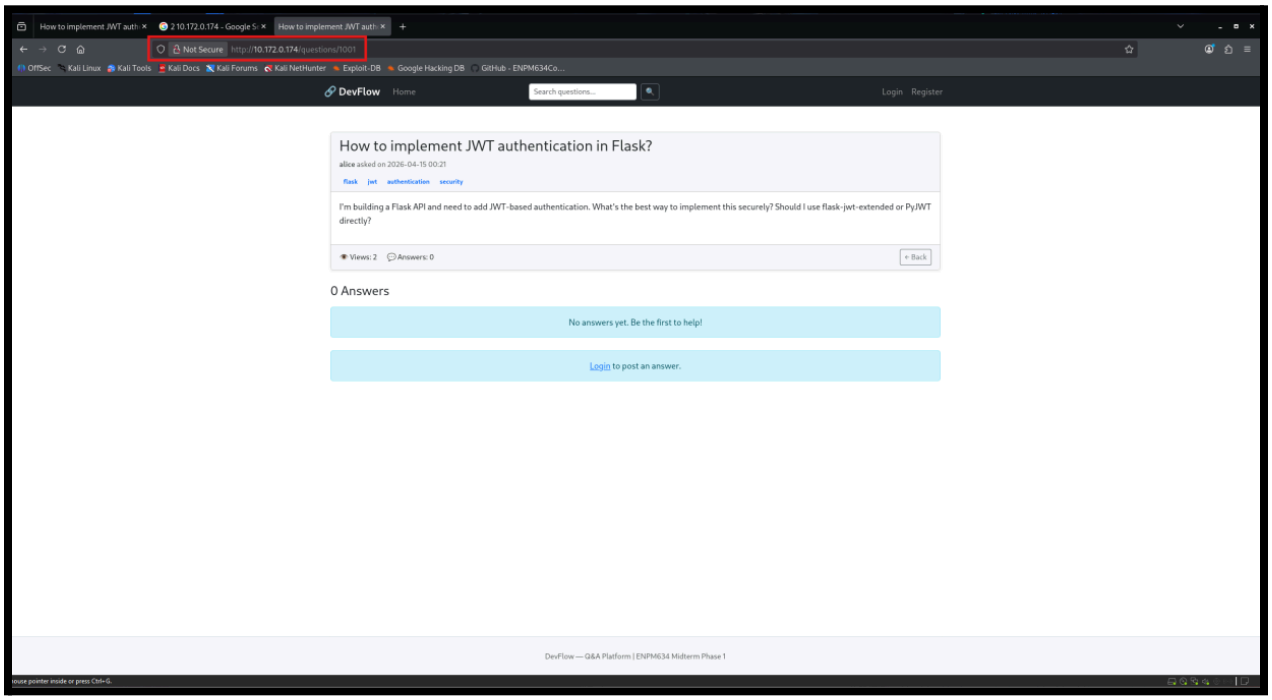
(1) After accessing DevFlow at `http://10.172.0.174`, we observed that question URLs follow a predictable sequential integer pattern (e.g., `http://10.172.0.174/questions/1001`).

(2) We wrote a Python enumeration script (i.e., `Soup.py`) using requests and BeautifulSoup to iterate through question IDs 1-1000 and extract their card body content; then ran the script via python3 `Soup.py` from our terminal and observed it cycling through all question IDs.

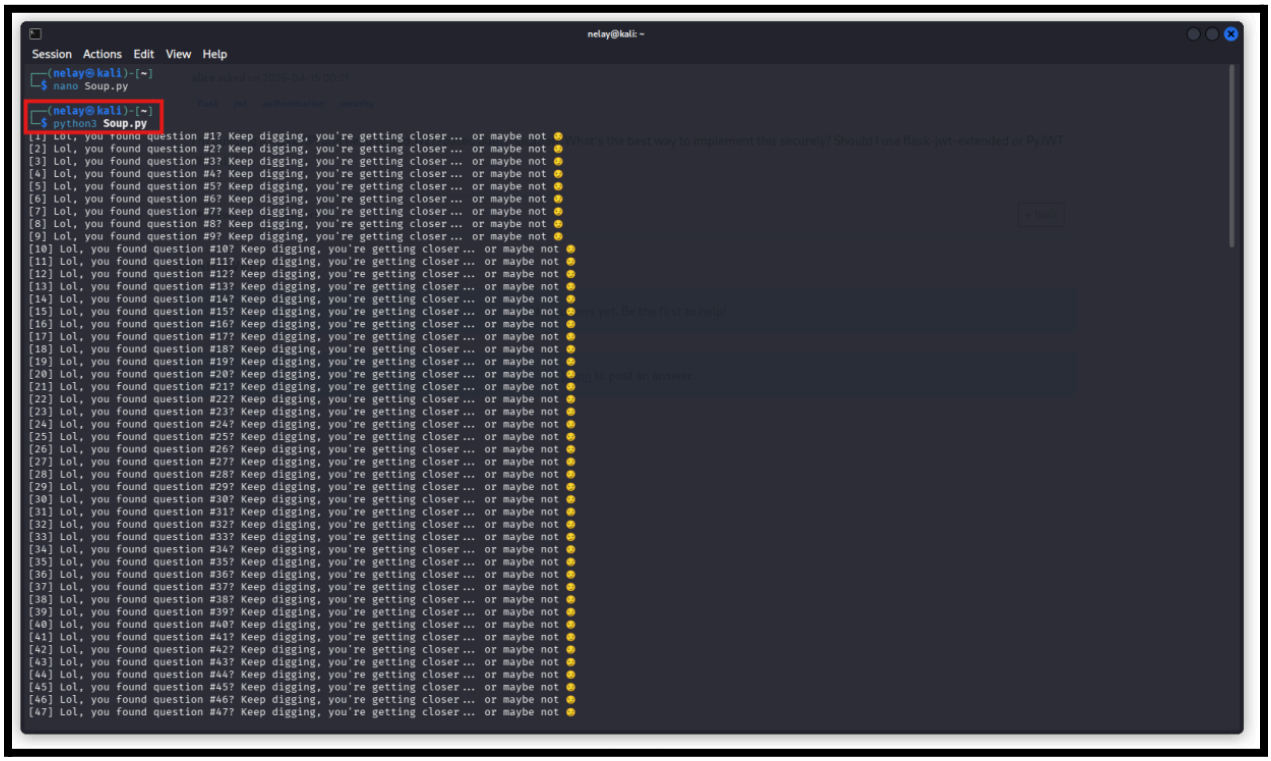
(4) At question ID 634, the script returned private content belonging to user alice, confirming the IDOR and revealing the flag: `ENPM634{idor_private_question_access}`.

(5) Navaging directly to `http://10.172.0.174/questions/634` in the browser and confirm the private question and flag were fully visible without authentication.

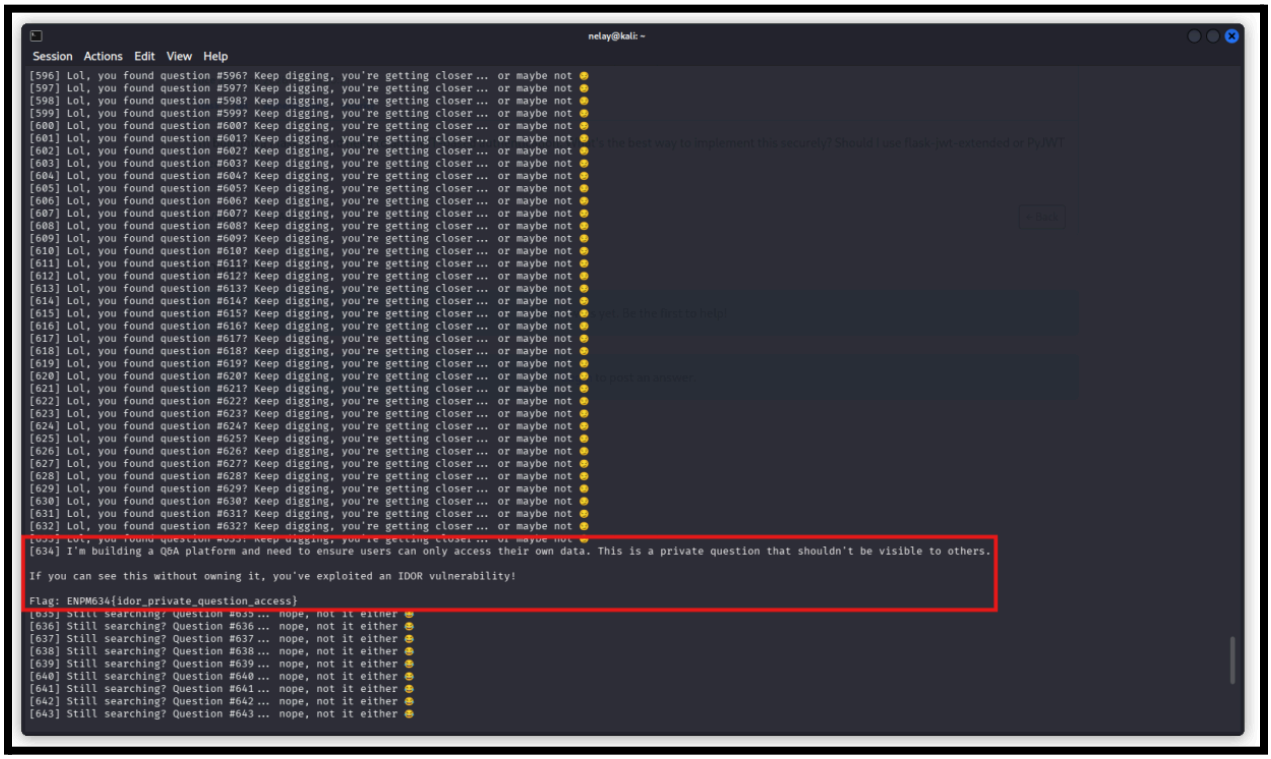
Screenshot #1: Public question page showing sequential URL pattern



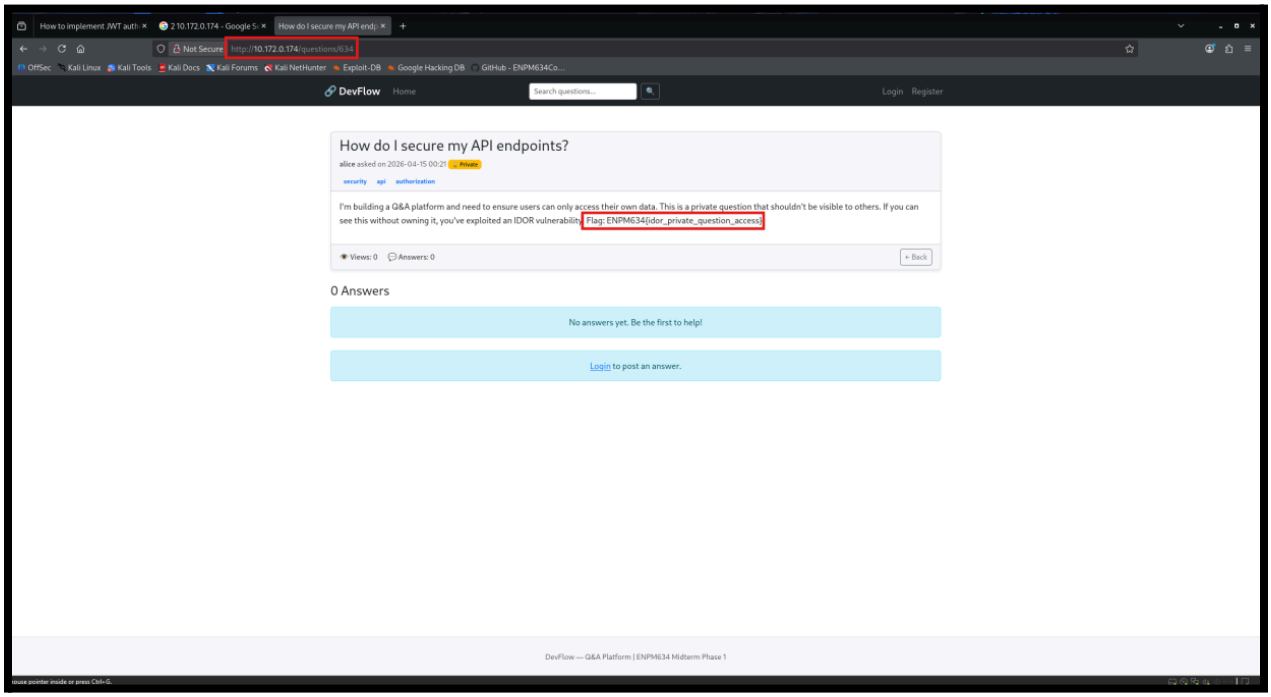
Screenshot #2: Soup.py enumeration script running, early results



Screenshot #3: Script output revealing private question at ID 634 with flag



Screenshot #4: Browser confirming private question and flag at /questions/634



Screenshot #5: Nano Soup.py as the secondary IDE

Impact

An unauthenticated attacker can enumerate all question IDs and access any private content on the platform without owning or being authorized to view it. This breaks the confidentiality of all private posts and exposes sensitive user data to the public. If this was a real world production application, then a full data dump of every private question on the platform is trivially achievable from an attacker with a simple enumeration script.

Remediation

The application must enforce server-side object-level authorization checks on every object access request, verifying that the currently authenticated user owns or has explicit permission to view the requested resource (i.e., owner match, role check, visibility flag). Sequential integer IDs should be replaced with non-guessable identifiers such as UUIDs.

Tools: Kali Linux, Firefox Browser, Python

‘Soup.py’ Source Code

```
import requests
from bs4 import BeautifulSoup

url = "http://10.172.0.174/questions/"

for i in range(1, 1001):
    response = requests.get(f"{url}/{i}")

    if response.status_code == 200:
        # Parse HTML instead of JSON
        soup = BeautifulSoup(response.text, 'html.parser')

        # Extract the card-body content
        card = soup.find(class_="card-body")

        if card:
            print(f"[{i}] {card.get_text(strip=True)}")
        else:
            print(f"[{i}] No card-body found")
    else:
        print(f"[{i}] Status: {response.status_code}")
```

Flag #4 - Challenge Summary

Name: JWT Token Forgery
Category: Web
Severity: High
Description: A guest user's token was found allowing for the forgery of different tokens.

Steps to Reproduce

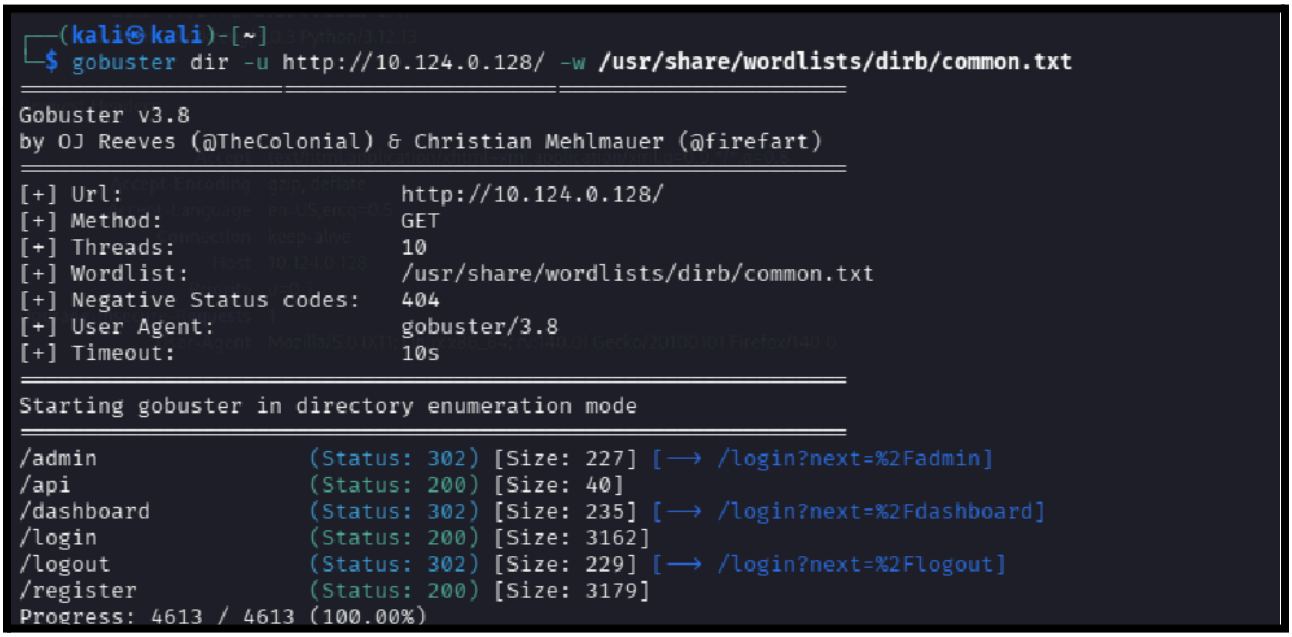
(1) Gobuster was used with the common.txt and big.txt wordlists to enumerate the directories accessible from the web server. JSON files were found at `http://10.175.0.55/api/dev` and `http://10.175.0.55/api/dev/token` with hints for exploitation as well as a JWT token for a guest account.

(2) The JWT token for the guest account was decoded using jwt.io revealing various plaintext data fields. A JWT secret is required to sign the JWT token. Another Python script was used to search for possible secret strings within the source code of the webpage.

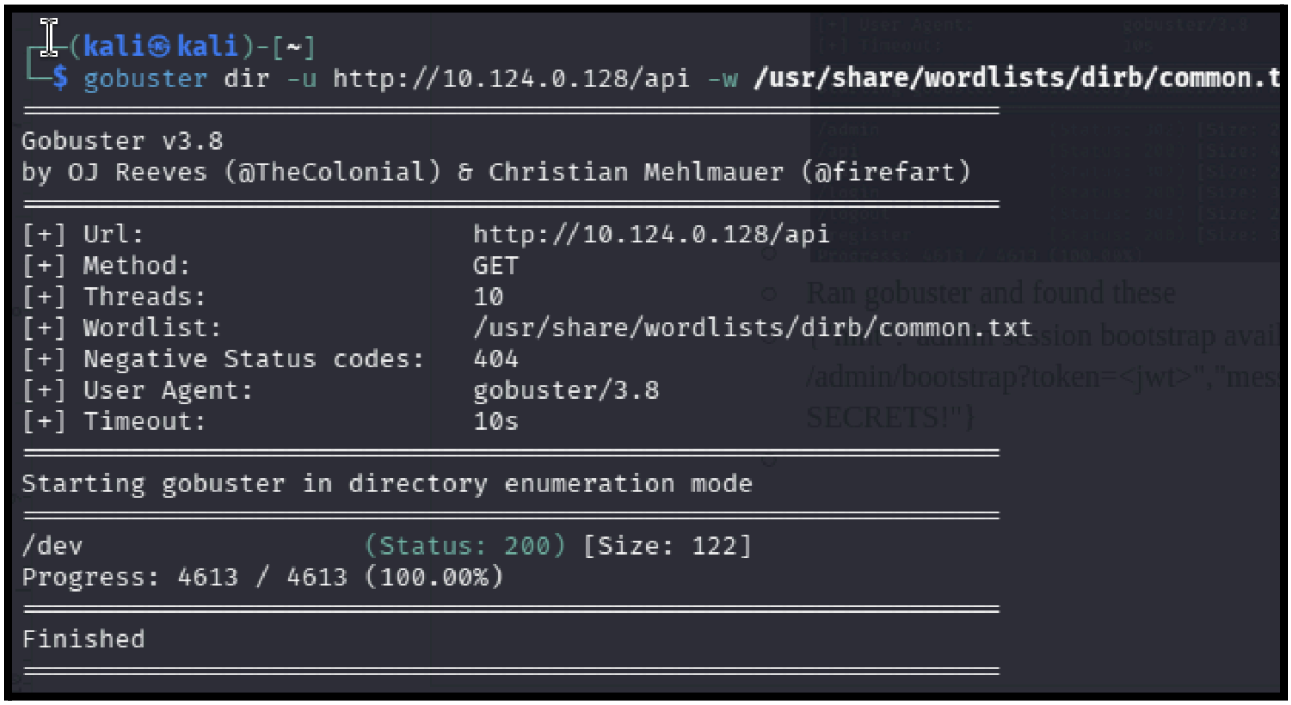
(3) After looking through the dumped source code, the secret signature was found which was then used to modify and sign a modified version of the JWT token. In the modified version the “sub” and “role” fields are both changed to admin.

(4) After the new JWT is created using the modified fields and the secret signature, it is passed into the website using the below URL giving us access to the admin page and the following flag: ENPM634{weak_jwt_secret_admin_forgery}

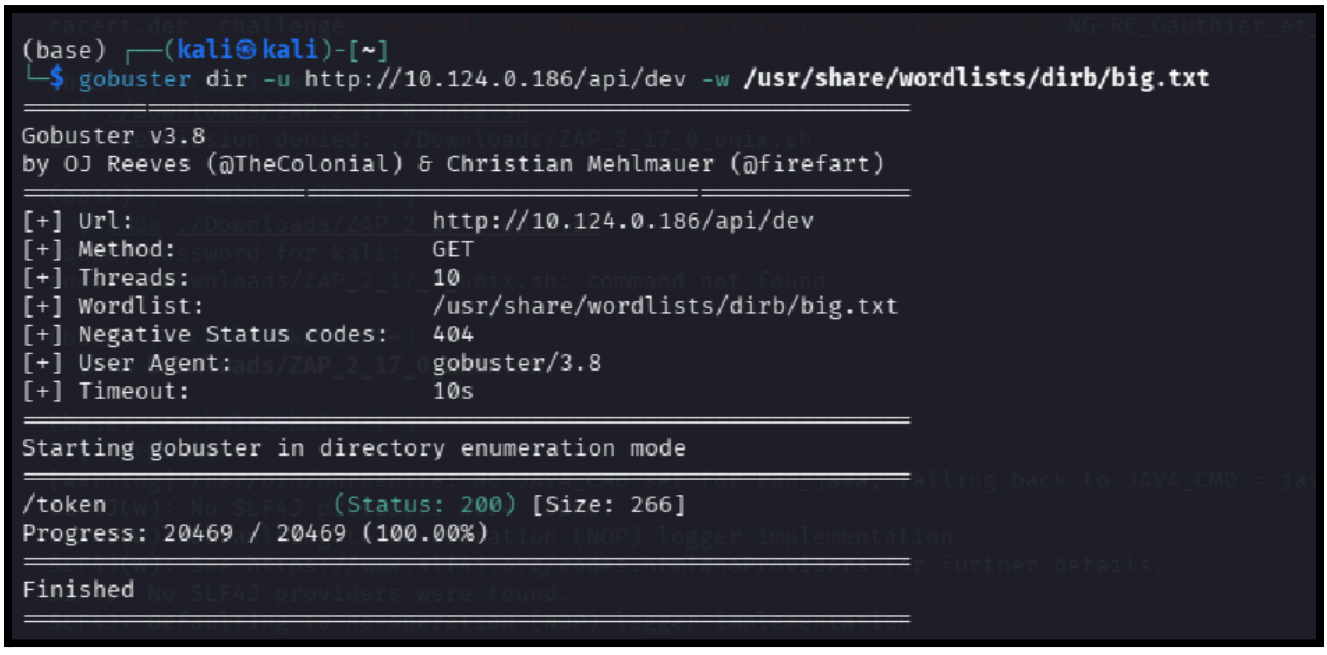
Screenshot #1: Initial scan of gobuster on the website



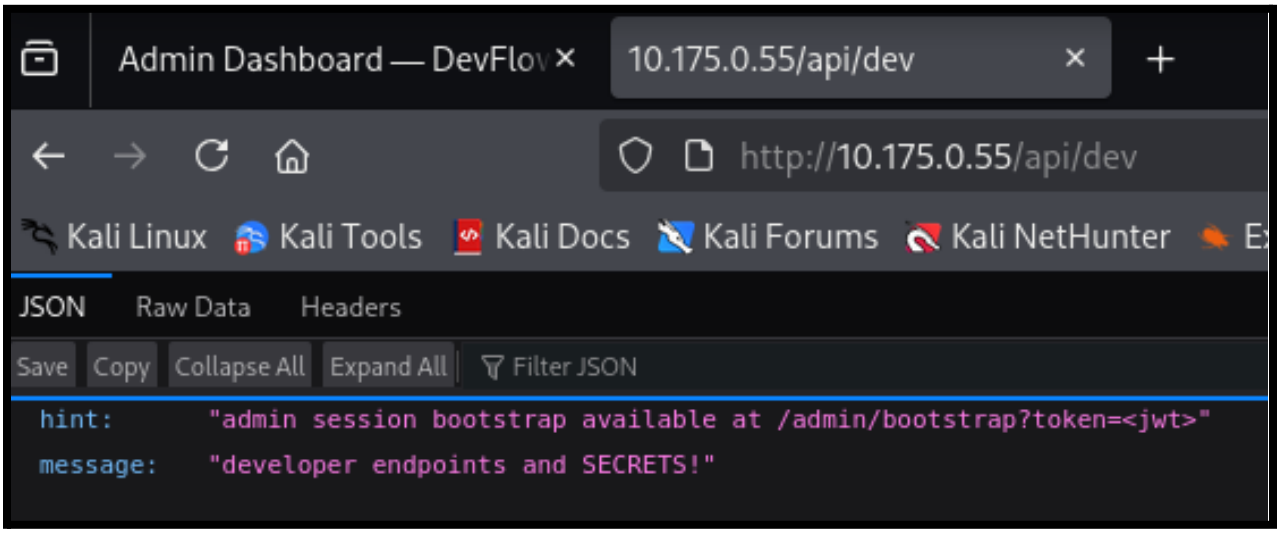
Screenshot #2: Gobuster finding the /api/dev directory



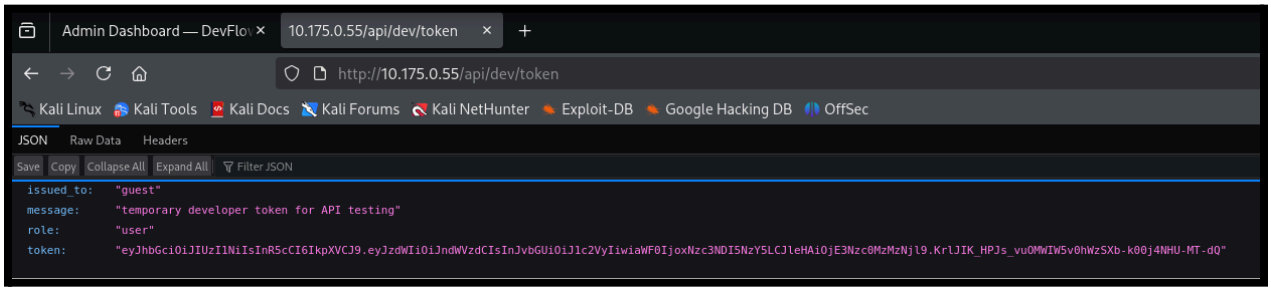
Screenshot #3: Gobuster run locating the /api/dev/token directory



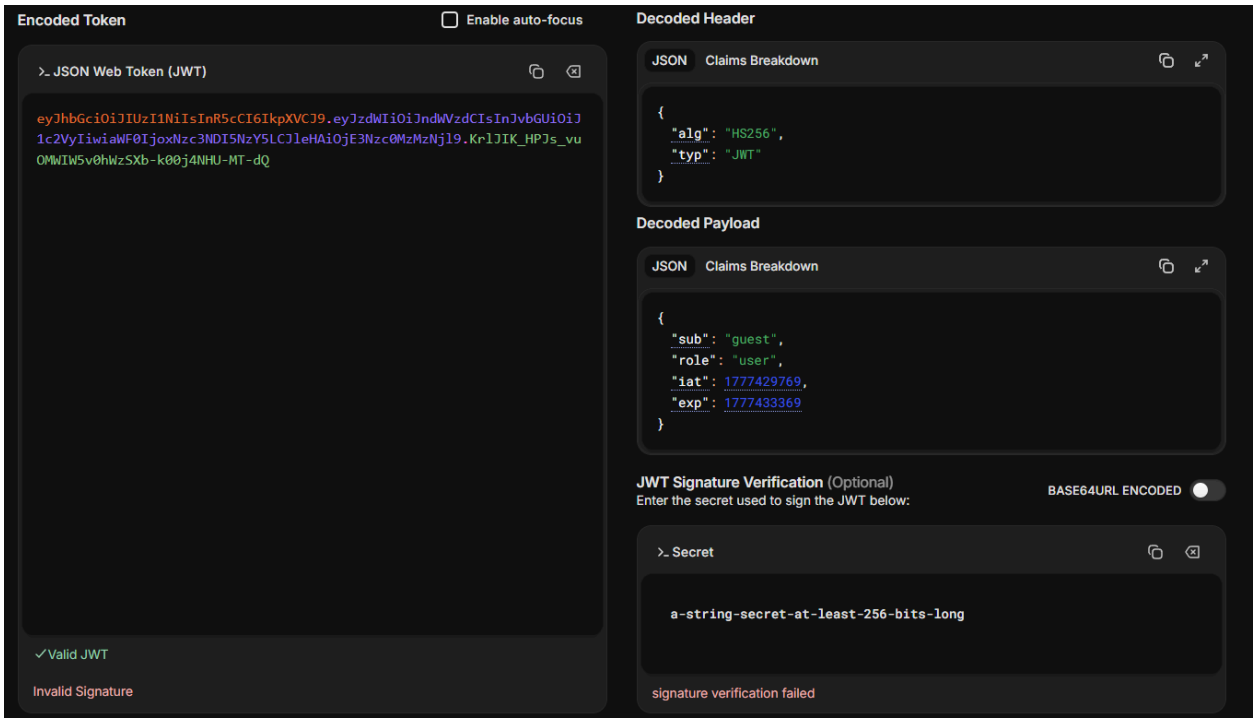
Screenshot #4: Contents of the /api/dev JSON



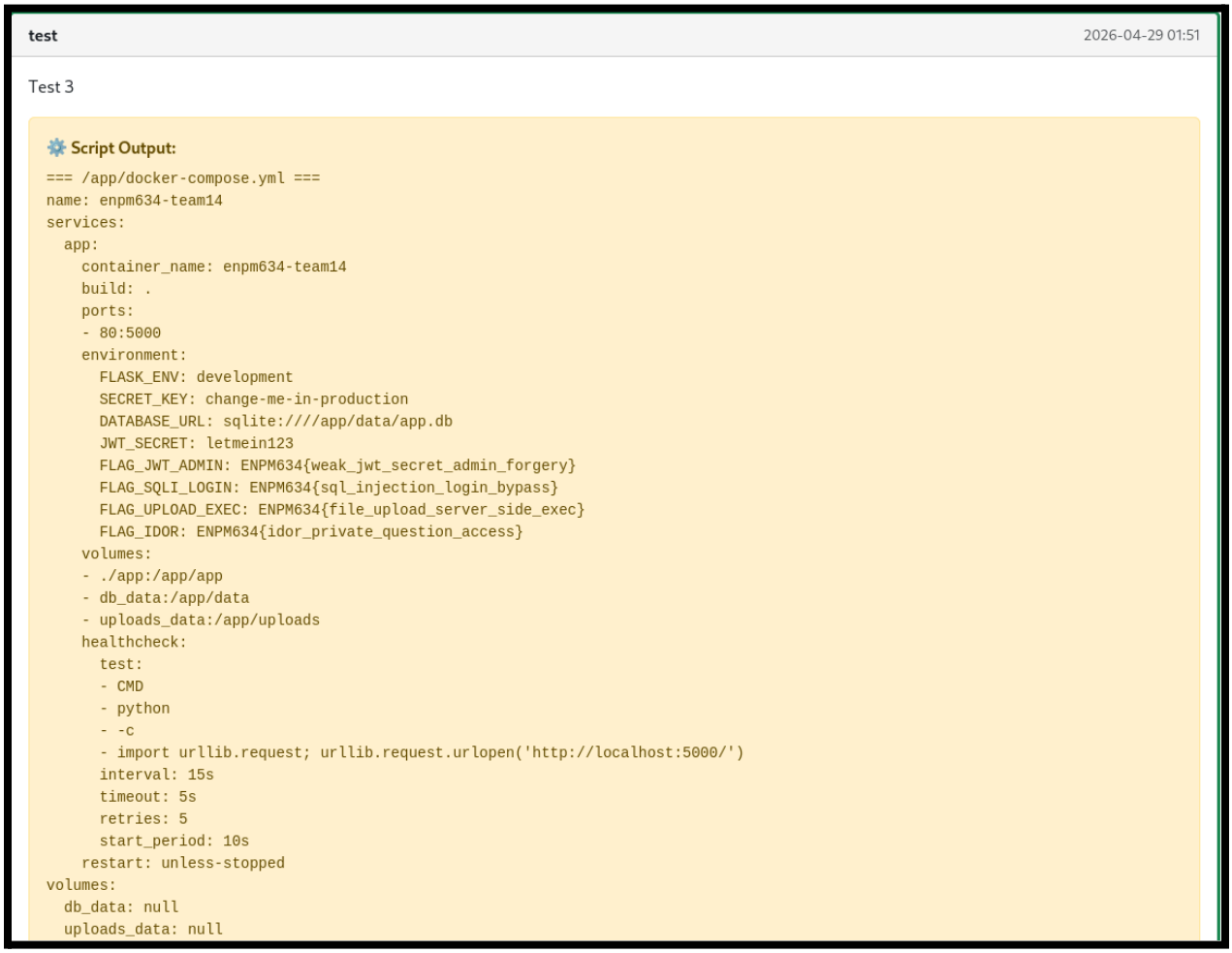
Screenshot #5: Contents of the api/dev/token JSON



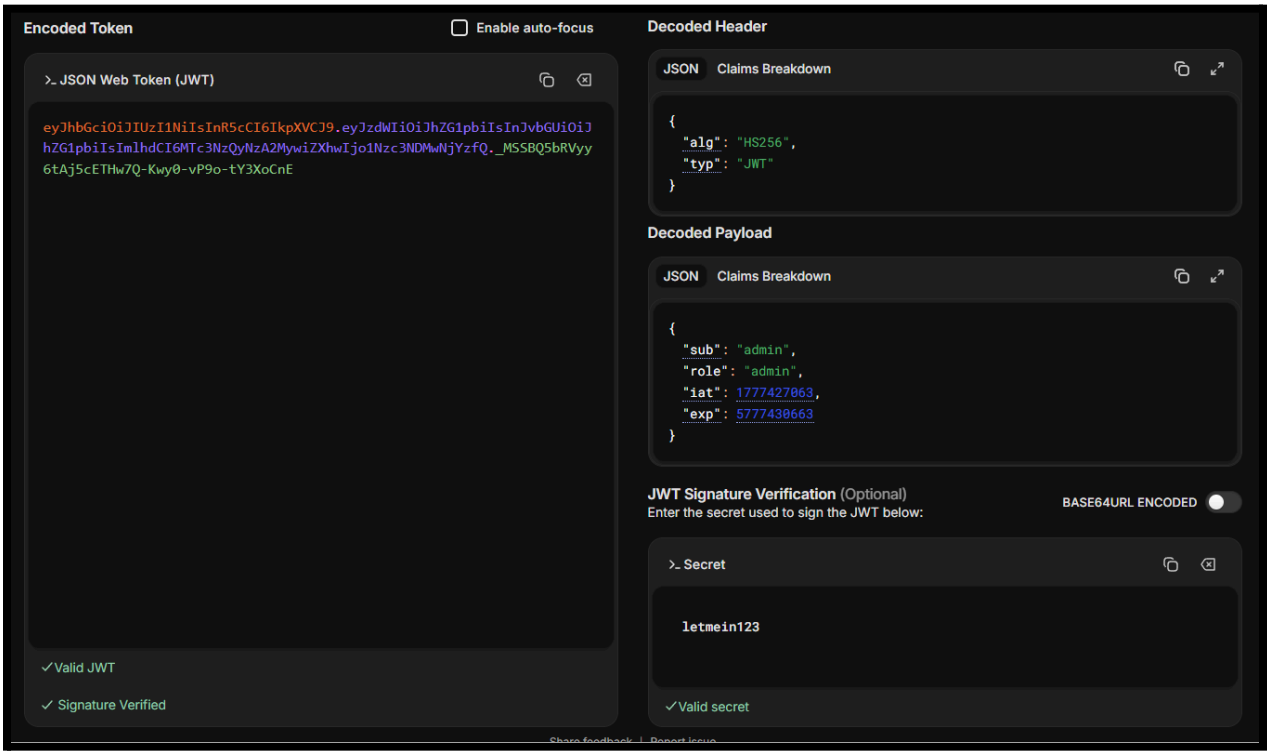
Screenshot #6: JWT token decoded



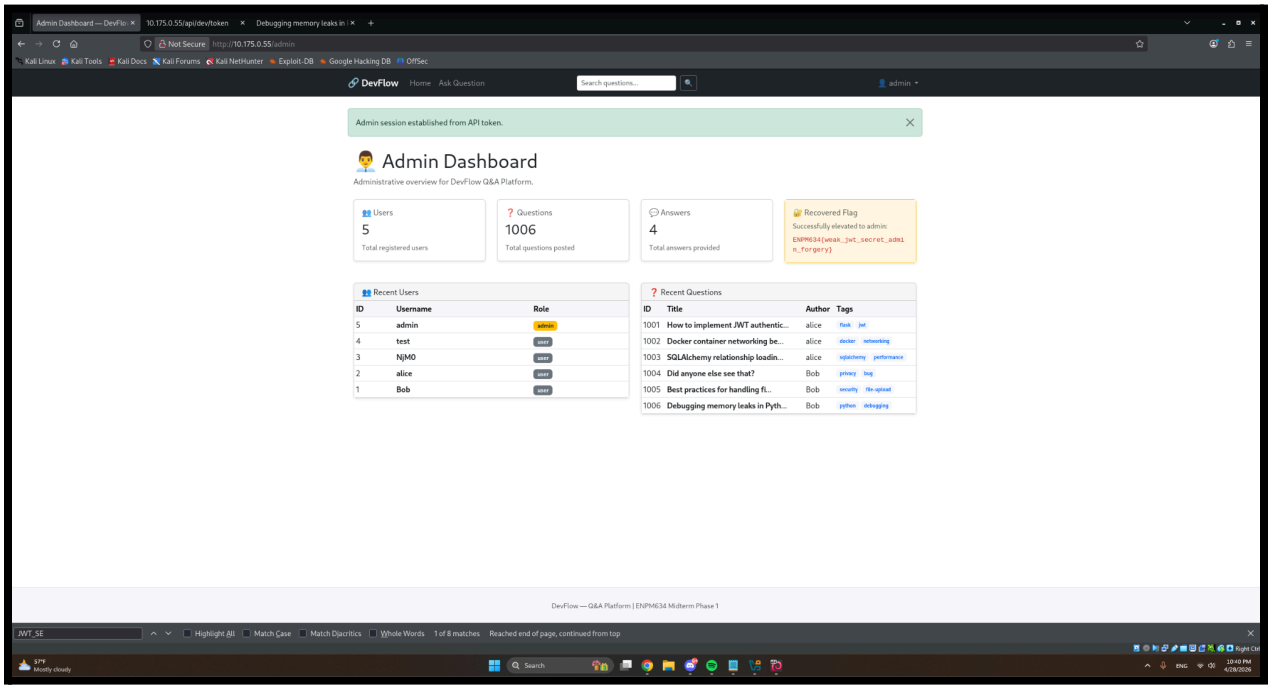
Screenshot #7: JWT secret found



Screenshot #8: JWT token forged for admin user



Screenshot #9: Login to admin page using JWT token



Impact

An attacker can steal an existing user's JWT token, decode it, analyze it, and possibly gain access to other accounts, including privileged accounts like the admin without knowing any credentials. With this if the attacker makes the right modifications and finds the secret signature for the token, they can arbitrarily access any account using the token system.

Remediation

The most obvious form of remediation is to avoid exposing anyone’s JWT token, as well as to hide the JWT signature. If the attacker doesn’t know how to craft the tokens, they won’t be able to forge them. Likewise if the signature can’t be obtained by them, they also won’t be able to forge a JWT token.

Tools: Kali Linux, Firefox Browser, Gobuster, `jwt.io`, Python

```
import os

for root, dirs, files in os.walk('/app'):
    for f in files:
        path = os.path.join(root, f)
        try:
            content = open(path).read()
            if any(x in content.lower() for x in ['secret', 'jwt', 'bootstrap']):
                print(f"\n=== {path} ===")
                print(content)
        except:
            pass
```

TOKEN

http://10.175.0.55/admin/boostrap?token=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJhZG1pbiIsInJvbGUiOiJhZG1pbiIsImlhdCI6MTc3NzQyNzA2MywiZXhwIjo1Nzc3NDMwNjYzfQ._MSSBQ5bRVyy6tAj5cETHw7Q-Kwy0-vP9o-tY3XoCnE

Contribution Statement

Contributors: Aron Sipos, Nelay Sharma, and Sheldon Douglas

Individual Contributions

- **Aron S.**: Discovered flags #2, #3, and forged a JWT to discover flag #4.
- **Nelay S**.: Compiled penetration testing report and ensured each vulnerability had a reproducible proof-of-concept.
- **Sheldon D.**: Discovered flag #1 and enumerated web server to discover a hidden JWT that assisted with flag #4.

Conclusion

The DevFlow Q&A application showed a range of serious vulnerabilities spanning from authentication, file handling, access control, token security, session management, input validation, and sensitive data exposure. The most critical issue is the server side file upload execution as it allows an unauthenticated attacker to run arbitrary code directly on the server, rendering all other controls irrelevant.

Close behind it is the SQL injection login bypass, and the weak JWT secret which compounds the risk significantly across all privilege levels, by providing multiple independent paths to full administrative access The IDOR vulnerability further demonstrates that the application lacks any meaningful authorization enforcement at the resource level, which is fundamental security failure, especially in a real world context where an attacker could silently exfiltrate every private post and sensitive user record on the platform, if given this was an enterprise level internal Q&A application.

Collectively, these findings reflect an application that was built without a security-first mindset and requires immediate foundational remediation starting with parameterized queries, upload sandboxing, object-level authorization checks, and secrets management, before it should be considered for any full-scale production deployment as not patching these vulnerabilities would lead to regulatory, reputational, and financial fines and penalties as it is in direct violation of baseline OWASP Top 10 security standards.